

PRAKTIKUM BASIS DATA

MODUL 11

(NoSQL_Key_Value_Database)



Oleh:

Danu Warisman

103122400041

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

SOAL 1: Analisis Kebutuhan Database

Davviz membangun platform streaming musik Spotifine dengan jutaan pengguna aktif. Database harus tahan banting (fault-tolerant) dan tidak mati meskipun satu server mengalami gangguan.

Atribut yang dirancang:

- **Artist ID:** ID unik untuk setiap musisi (Primary Key) - UUID
- **Artist Name:** Nama lengkap musisi atau nama panggung - TEXT
- **Followers:** Jumlah pengikut - INT
- **Origin:** Negara asal musisi - TEXT
- **Genres:** Daftar aliran musik (multiple genres per musisi) - SET<TEXT>

Mengapa Cassandra? Cassandra dipilih karena:

- High availability dan fault tolerance (dapat handle node failure)
- Scalability horizontal untuk jutaan pengguna
- Support untuk tipe data collection (SET) yang sempurna untuk multiple genres
- Eventually consistent model cocok untuk aplikasi real-time

SOAL 2: IMPLEMENTASI

a. CREATE KEYSPACE

Query:

```
cqlsh> CREATE KEYSPACE spotifine_0802
... WITH REPLICATION = {
...   'class' : 'SimpleStrategy',
...   'replication_factor' : 1
... };
cqlsh>
```

Penjelasan:

- SimpleStrategy: Strategi replikasi sederhana untuk development/single-datacenter
- replication_factor 1: Data disimpan di 1 node (untuk testing environment)

Output yang seharusnya muncul di cqlsh:

```
cqlsh> CREATE KEYSPACE spotifine_0802
... WITH REPLICATION = {
...   'class' : 'SimpleStrategy',
...   'replication_factor' : 1
... };
cqlsh>
```

Menggunakan Keyspace:

```
cqlsh> USE spotifine_0802;_
```

Output:

```
cqlsh> USE spotifine_0802;
cqlsh:spotifine_0802> _
```

b. CREATE TABLE

Query:

```
cqlsh:spotifine_0802> CREATE TABLE musician_data (
...   artist_id UUID PRIMARY KEY,
...   artist_name TEXT,
...   followers INT,
...   origin TEXT,
...   genres SET<TEXT>
... );
```

Penjelasan:

- artist_id UUID PRIMARY KEY: Partition key yang unik untuk setiap musisi, menggunakan UUID untuk guaranteed uniqueness
- artist_name TEXT: Nama musisi dalam format text
- followers INT: Integer untuk menampung jumlah pengikut (hingga 2.147.483.647)
- origin TEXT: Negara asal musisi
- genres SET<TEXT>: Set collection untuk multiple genres tanpa duplikat

Output:

```
cqlsh:spotify_0802> CREATE TABLE musician_data (
...     artist_id UUID PRIMARY KEY,
...     artist_name TEXT,
...     followers INT,
...     origin TEXT,
...     genres SET<TEXT>
... );
```

c. INSERT DATA (3 Musisi)

Data Musisi 1:

```
... );
cqlsh:spotify_0802> INSERT INTO musician_data (artist_id, artist_name, followers, origin, genres)
... VALUES (uuid(), 'The Weeknd', 80000000, 'Canada', {'R&B', 'Hip-Hop', 'Synthwave'});
```

Data Musisi 2:

```
... VALUES (uuid(), 'The Weeknd', 80000000, 'Canada', {'R&B', 'Hip-Hop', 'Synthwave'});
cqlsh:spotify_0802> INSERT INTO musician_data (artist_id, artist_name, followers, origin, genres)
... VALUES (uuid(), 'Taylor Swift', 90000000, 'United States', {'Pop', 'Country', 'Folklore'});
```

Data Musisi 3:

```
cqlsh:spotify_0802> INSERT INTO musician_data (artist_id, artist_name, followers, origin, genres)
... VALUES (uuid(), 'Bad Bunny', 85000000, 'Puerto Rico', {'Reggaeton', 'Trap Latino', 'Pop'});
```

d. SELECT SEMUA DATA

Query:

```
cqlsh:spotify_0802> SELECT * FROM musician_data;
```

Output yang seharusnya muncul:

```
cqlsh:spotify_0802> SELECT * FROM musician_data;
```

artist_id	artist_name	followers	genres	origin
4a357b4a-5422-4be4-a409-613e790e7e87	Bad Bunny	85000000	{'Pop', 'Reggaeton', 'Trap Latino'}	Puerto Rico
c18d6521-f336-401f-bd1e-6c949b9084d9	Taylor Swift	90000000	{'Country', 'Folklore', 'Pop'}	United States
918121d0-0226-4093-8d46-26f14942db86	The Weeknd	80000000	{'Hip-Hop', 'R&B', 'Synthwave'}	Canada

```
(3 rows)
cqlsh:spotify_0802>
```

SOAL 3: Keuntungan Menggunakan SET<TEXT> untuk Genres

Keuntungan Cassandra SET<TEXT> vs RDBMS:

1. Denormalisasi & No JOIN Required:

- **Cassandra:** Genres langsung nested dalam satu row musisi. Query SELECT * FROM musician_data langsung mendapat semua info termasuk genres tanpa JOIN.
- **RDBMS:** Harus buat 3 tabel: musicians, genres, dan musician_genres (junction table). Query memerlukan 2-3 JOIN untuk mendapat data lengkap.
- **Keuntungan:** Cassandra lebih simple dan tidak perlu join logic yang kompleks.

2. Query Performance (Lower Latency):

- **Cassandra:** Satu query SELECT langsung ke satu row, tidak ada seek ke multiple tables. Latency tetap konsisten.
- **RDBMS:** JOIN operation meningkatkan latency, terutama dengan jutaan records.
- **Keuntungan:** Cassandra lebih cepat untuk read-heavy workload seperti streaming app.

3. Horizontal Scalability:

- **Cassandra:** SET dapat handle ratusan atau ribuan genre per musisi tanpa performance degradation. Cassandra dirancang untuk horizontal scaling.
- **RDBMS:** JOIN dengan banyak records (millions) menjadi bottleneck. Scaling memerlukan complex sharding.
- **Keuntungan:** Cassandra scale-out lebih mudah untuk aplikasi dengan millions of users.

4. No Duplicate Genres Automatically:

- **Cassandra:** SET secara otomatis mencegah duplicate. Jika insert genre yang sama 2x, hanya tersimpan 1x.
- **RDBMS:** Harus implement unique constraint di junction table atau handle di application layer.
- **Keuntungan:** Cassandra lebih clean dan error-free.

5. Flexible & Easy to Modify:

- **Cassandra:** Mudah add/remove genre tanpa UPDATE kompleks. UPDATE musician_data SET genres = genres + {'Electronic'} WHERE artist_id = xxx
- **RDBMS:** DELETE/INSERT ke junction table, lebih verbose.
- **Keuntungan:** Code lebih sederhana dan less error-prone.

6. High Availability Built-in:

- **Cassandra:** Data replicated across multiple nodes. Jika satu node down, data masih available. SET collection support ini built-in untuk distributed system.
- **RDBMS:** Replication lebih kompleks dan memerlukan master-slave setup.
- **Keuntungan:** Cassandra cocok untuk aplikasi yang butuh 24/7 uptime seperti Spotifine.

7. Eventual Consistency Model:

- **Cassandra:** Cocok untuk streaming app yang prioritaskan availability over strict consistency. Jika user follow artist, genre info update eventually (tapi aplikasi tetap responsive).
- **RDBMS:** ACID compliance memerlukan lock, potentially slow untuk high-concurrency app.
- **Keuntungan:** Better user experience (fast response) untuk musik streaming.